NAME : V.G. ABBIRAMY
REG NO : 192524017
COURSE CODE : CSAI408
COURSE NAME : Compiler Design

1. Translation Scheme for Assignment Statement :

$S \rightarrow id = E$

{emit (id.lexeme || "=" || E.place)}

$E \rightarrow E1 + E2$

{E.place = newtemp();
emit (E.place || "=" || E1.place || "+" || E2.place)}

$E \rightarrow E1 - E2$

{E.place = newtemp();
emit (E.place || "=" || Er.place || "=" || E2.place)}

$E \rightarrow E_1 * E_2$

{E.place = newtemp();
emit (E.place || "=" || E1.place || "*" || E2.place)}

$E \rightarrow E1 / E2$

{E.place = newtemp();
emit (E.place || "=" || E1.place || "/" || E2.place)}

$E \rightarrow (E1)$

{E.place = E1.place}

$E \rightarrow id$

{E.place = id.lexeme}

For $x = (a+b) * (c+d)$,

TAC :

```
t1 = a + b
t2 = c + d
t3 = t1 * t2
x = t3
```

2. Boolean Expression :

Given :

$(a > b) \&\& (c < d) \,||\, (e < f)$

i) Precedence & Associativity :

&& has higher precedence than ||.

So the expression is :

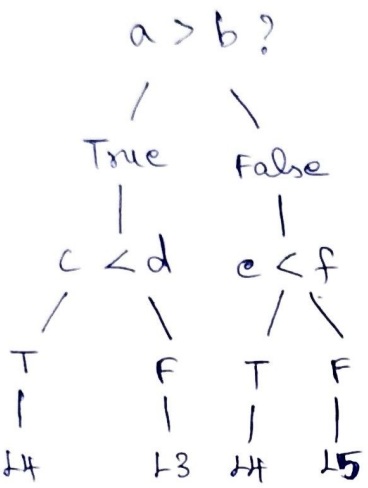
$((a > b) \&\& (c < d)) \,||\, (e < f)$

Both && and || are evaluated from left to right when they have the same precedence.

ii) TAC :

```
if a > b goto L1
goto L3
L1 : if c < d goto L4
     goto L3
L3 : if e < f goto L4
     goto L5
L4 : // True
L5 : // False
```

Control flow:



3. Symbol Table:

Given:

```
int a, b[10];
float c,d;
```

Name	Type	Size	Offset
a	int	4 bytes	0
b	int[10]	40 bytes	4
c	float	4 bytes	44
d	float	4 bytes	48

Intermediate representation:

a: int

b: array(10, int)

c: float

d: float

Total : 52 bytes

4. i) TAC for while-if fragment:

Given:

```
while (A<C && B>D) do
  if (A==1) then
     C = C+1
  else
     while (A<=D) do
        A = A+B
```

TAC:

```
L1 : if A<C goto L2
     goto L8
L2 : if B>D goto L3
     goto L8
L3 : if A==1 goto L4
     goto L6
L4 : t1 = C+1
     C = t1
     goto L1
L6 : if A<=D goto L7
     goto L1
L7 : t2 = A+B
     A = t2
     goto L6
```

ii) TAC for C program:

Given array & loop program

```
i = 1
L1 : if i<=10 goto L2
     goto L5
L2 : t1 = i*4
     a[t1] = 0
     t2 = i+1
     i = t2
     goto L1
```

∴ Here i*4 is used because each integer occupies 4 bytes.

patching:

given:

```
if (a<b && c<d)
    x = a+b;
else
    x = c+d;
```

Before backpatching:

```
1: if a<b goto _
2: goto _
3: if c<d goto _
4: goto _
5: t1 = a+b
6: x = t1
7: goto _
8: t2 = c+d
9: x = t2
```

After backpatching:

```
1: if a<b goto 3
2: goto 8
3: if c<d goto 5
4: goto 8
5: t1 = a+b
6: x = t1
7: goto 10
8: t2 = c+d
9: x = t2
10: end
```